Estensione di una scena OpenGL a classi

Esercitazione di laboratorio – C++ / GLFW / GLAD / GLM / Dear ImGui

Obiettivo. Dopo aver studiato il template fornito, estendere l'applicazione senza alterarne la separazione delle responsabilità: generazione dei dati geometrici sulla CPU, gestione delle risorse OpenGL nella classe `Geometry`, organizzazione degli oggetti in `Scene`, disegno in `Renderer` e controllo interattivo tramite `Gui`.

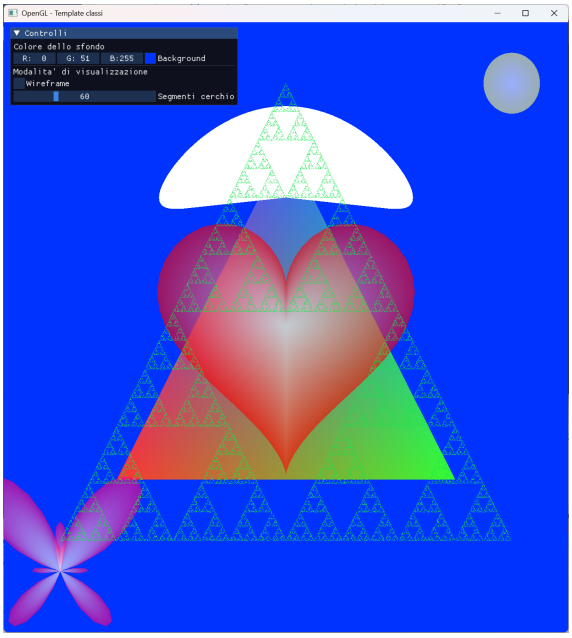


Figura 1: Esempio di scena OpenGL con geometrie differenti e pannello di controllo Dear ImGui.

1. Analisi preliminare del template

1. Individuare, nei file `.h` e `.cpp`, i compiti delle classi `ShapeGenerator`, `Geometry`, `Scene`, `Renderer`, `Shader` e `Gui`, nonché della struttura `MeshData`.

2. Ricostruire il percorso seguito dai dati del triangolo: `createTriangle()` → `MeshData` → `Scene::add()` → `Geometry::upload()` → `Renderer::drawScene()`.

2. Generazione di nuove forme sulla CPU

1. Partendo dall'implementazione di `createTriangle()`, completare o analizzare le funzioni di `ShapeGenerator` che costruiscono una circonferenza, un cuore, una farfalla e una luna. Ogni funzione deve restituire un `MeshData` con posizioni, colori e tipo di primitiva coerenti.

2. Campionare le curve parametriche con un numero configurabile di segmenti, usando `std::vector`, `reserve()` e `push_back()`. Verificare che ogni posizione abbia il colore corrispondente.

3. Aggiungere ciascuna geometria alla scena una sola volta durante l'inizializzazione e scegliere una primitiva OpenGL appropriata alla forma. Motivare l'eventuale uso di `GL_TRIANGLE_FAN` oppure `GL_LINE_STRIP`.

3. Sierpinski gasket

1. Generare il frattale mediante il *Chaos Game*: fissati tre vertici T_0, T_1, T_2, scegliere casualmente T e iterare $p_{k+1} = \frac{1}{2}(p_k + T)$.
2. Salvare i punti e i colori nei vector del `MeshData`; disegnare con `GL_POINTS` e un numero di iterazioni sufficiente a riconoscere la struttura frattale.
3. Evitare di chiamare `scene.add()` a ogni frame per una geometria che non cambia.

4. Geometrie dinamiche: animazione del cuore

1. Inserire il cuore nella scena come geometria dinamica.
2. Utilizzare il tempo restituito da `glfwGetTime()` per variare periodicamente le dimensioni del cuore.
3. Ricostruire le posizioni sulla CPU e aggiornare il VBO esistente tramite `Geometry::updatePositions()`, senza aggiungere un nuovo oggetto alla scena. Mantenere costante il numero di vertici durante questo aggiornamento.

5. Controlli Dear ImGui

1. Inserire nella classe `Gui` un selettore `ColorEdit3` per il colore di sfondo e una checkbox per la modalità *Fill/Wireframe*.
2. Esporre al `main` i valori tramite metodi pubblici di sola lettura; utilizzare tali valori nei metodi `Renderer::beginFrame()` e `Renderer::setWireframe()`.
3. Aggiungere almeno un campo numerico editabile e un pulsante `Reset`, chiarendo quali variabili private della GUI vengono modificate.
4. Verificare l'ordine di esecuzione: nuovo frame GUI, costruzione dei widget, rendering della scena, ripristino `GL_FILL`, rendering della GUI e scambio dei buffer.

6. Slider per il numero di segmenti del cerchio

1. Aggiungere un controllo `SliderInt` con un intervallo valido (ad esempio da 3 a 60 segmenti) e un metodo pubblico per leggere il valore corrente.
2. Rilevare nel `main` quando il valore cambia; solo allora ricostruire i dati del cerchio mediante `ShapeGenerator::createCircle()`.
3. Conservare centro, raggi e colori originari durante la ricostruzione: lo slider deve modificare soltanto la discretizzazione.
4. Aggiornare la `Geometry` già presente nella scena, senza richiamare `scene.add()`. Poiché il numero di vertici cambia, garantire che i VBO abbiano capacità sufficiente: è ammessa la ricreazione mediante `setData()` e `upload(true)`, oppure una gestione esplicita della riallocazione.

7. Blending e trasparenza delle geometrie

1. Abilitare il *blending* OpenGL durante l'inizializzazione della scena mediante `glEnable(GL_BLEND)` e configurare i fattori di composizione con `glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA)`.

2. Utilizzare il quarto canale dei colori `glm::vec4` per assegnare ad almeno due geometrie un valore di trasparenza $0 < \alpha < 1$. Confrontare il risultato con il caso opaco ($\alpha = 1$).

3. Disporre le geometrie in modo che si sovrappongano parzialmente e verificare visivamente come il colore risultante dipenda dai colori delle figure, dai valori di α e dall'ordine di disegno.

4. Spiegare la differenza tra il canale alpha dei colori dei vertici, il colore usato per cancellare il framebuffer e l'abilitazione effettiva del blending. Verificare che il fragment shader trasmetta il canale alpha al colore di uscita.

5. Motivare perché, nella scena 2D del progetto, l'ordine di disegno degli oggetti trasparenti può cambiare il risultato. Non è richiesto implementare un ordinamento generale per scene 3D.

Vincolo architetturale. La classe `Gui` gestisce lo stato dei controlli; `ShapeGenerator` costruisce i dati CPU; `Geometry` gestisce VAO e VBO; `Scene` contiene gli oggetti; `Renderer` coordina il disegno. Evitare di trasferire le responsabilità di una classe a un'altra.